

PROJECT DOCUMENTATION

1. Title

AI Podcast Summary – Automated Podcast Transcription and Topic Segmentation

2. Overview

The **AI Podcast Summary** project is an AI-based system that automatically transcribes podcast audio files and organizes the content into meaningful topic-based segments. It enables users to quickly understand long podcast episodes by providing transcripts, keyword extraction, sentiment insights, and concise summaries, reducing the need to listen to the full audio.

3. Approach

The system follows a step-by-step processing pipeline:

1. Podcast audio files are collected as input.
 2. Audio preprocessing techniques are applied to enhance quality.
 3. Speech-to-text conversion generates textual transcripts.
 4. NLP techniques are used for keyword extraction and topic segmentation.
 5. Segment-wise summaries and keyword-based insights are generated.
 6. Results are stored and visualized through a simple user interface.
-

4. Tech Stack

- **Programming Language:** Python
 - **Web Framework:** Streamlit
 - **Audio Processing:** Librosa, PyDub
 - **Natural Language Processing:** NLTK, Scikit-learn
 - **Visualization:** Matplotlib, WordCloud
 - **Data Storage:** JSON
-

5. Project Structure

Folder & File Descriptions

input_audio/

Stores raw podcast audio files uploaded by the user.

processed_audio/

Contains cleaned and preprocessed audio files after noise reduction and normalization.

audio_transcripts/

Stores generated transcript files converted from audio using speech-to-text models.

audio_segment_keySearch_summary/

Contains topic-wise segmented summaries, extracted keywords, and related JSON outputs.

src/

Contains core logic and processing scripts of the project.

- **preprocessing.py**
Handles audio preprocessing tasks such as cleaning, normalization, and preparation for transcription.
 - **transcript.py**
Converts processed audio files into text using speech-to-text techniques.
 - **segment_keySearch.py**
Performs topic segmentation, keyword extraction, and summary generation from transcripts.
 - **__pycache__**
Stores compiled Python bytecode files (auto-generated).
-

app.py

Main Streamlit application file responsible for the user interface and integration of all modules.

config.py

Central configuration file that defines directory paths and project-level settings.

.gitignore

Specifies files and folders to be excluded from Git version control.

README.md

Provides a high-level description of the project, features, and usage instructions.

LICENSE

Defines the legal license under which the project is distributed.

6. Usage

Users can upload podcast audio files through the Streamlit interface to view transcripts, keyword clouds, topic-wise summaries, and sentiment insights in an interactive manner.

7. Troubleshooting

- Ensure audio files are in supported formats (MP3/WAV).
 - Verify correct folder paths in config.py.
 - Check transcript and summary folders if outputs are missing.
 - Ensure required Python libraries are installed.
-

8. Limitations and Challenges

- Transcription accuracy depends on audio clarity and background noise.
 - Topic segmentation may not perfectly capture conversational shifts.
 - Sentiment analysis has limitations in detecting sarcasm or mixed emotions.
 - Large audio files require higher processing time.
-

9. Future Scope

- Support for multi-language podcasts.
 - Integration of transformer-based summarization models.
 - Cloud deployment for scalability.
 - Real-time podcast transcription.
 - Interactive timeline-based topic navigation.
-

10. References

1. **OpenAI Whisper**
2. **Librosa Audio Processing Library**
3. **NLTK (Natural Language Toolkit)**
4. **Scikit-learn Documentation**
5. **Streamlit Documentation**
6. **WordCloud Library**
7. **Matplotlib Documentation**
8. **Speech and Language Processing**

